

Recursive Undecidability—An Exposition

James P. Jones

Originally published in *The American Mathematical Monthly*
81 (September 1974), 724–738

Corrected transcription

Editorial notice. Continuously revised edition: every discrepancy between the printed article and this edition is explained, with its justification, in `Papers/1974/jones1974_editorial_notes.md`; a dated log of revisions and checks is `Papers/EDITORIAL_NOTES.md`. Bracketed margin numbers mark the lines on which the pages of the original printing begin. This is a corrected transcription, not a facsimile. The historical narrative and numerical record table refer to 1974, not to the present day. Mathematical qualifications, repaired proof details, and textual changes are documented in the accompanying *Editorial notes*; added explanations should not be mistaken for verbatim wording of the original. The original 42 numbered references are preserved.

Abstract

This expository article contains an explanation of recursive unsolvability, examples of non-computable functions and an example of a two-person game in which neither player has an algorithm to win. The intention is to provide a simple explanation of the subject, comprehensible to the non-logician. The results presented here are especially suited to this purpose because their proofs may be based entirely upon simple combinatorial arguments which avoid the formal arithmetization usually necessary in recursive function theory.

1 Introduction

The discovery of proofs to the effect that certain mathematical problems are unsolvable has been [724] one of the more remarkable achievements of mathematics of the nineteenth and twentieth centuries. Consider the ancient Greek geometry problems, doubling the cube and trisecting an arbitrary angle with compass and straightedge. The search for solutions went on for over 2000 years. Then it was proved that what was sought did not exist. The history of attempts to solve the general fifth-degree polynomial equation is a similar story. Early in the nineteenth century Abel, Galois and Ruffini proved the impossibility of solving it by means of radicals. Somewhat later Lindemann proved the transcendence of π , thus disposing of the age-old problem of squaring the circle.

The classical unsolvability proofs were a rich source of new ideas for mathematics. The work of Galois would later inspire the entire development of modern algebra. But bold explorers did not wait long before taking the next step: proofs of unprovability.

In the first half of the nineteenth century Gauss, Bolyai and Lobachevsky developed non-Euclidean geometry. Later model constructions established the independence of the parallel postulate from the remaining axioms of an appropriate axiomatization of geometry. Assuming the consistency of the real-number theory used to construct the models, both the parallel postulate and its negation are consistent with those remaining axioms. Thus neither is provable from them.

In 1931 K. Gödel showed [13] that the theory of numbers also contains undecidable propositions. His work showed that incompleteness cannot simply be eliminated by adding axioms while retaining the relevant effectiveness and arithmetic-strength hypotheses. In its strengthened Gödel–Rosser form, the theorem applies to every consistent, effectively axiomatized theory that contains sufficient elementary arithmetic. In 1963–64, P. Cohen [7] established the independence results which, together with Gödel’s relative-consistency work, show that the axiom of choice and the generalized continuum hypothesis are independent of Zermelo–Fraenkel set theory, assuming that theory is consistent. Gödel had already obtained the relative consistency of these two propositions in 1940 [12]. Thus these two famous problems were shown to be undecidable in axiomatic set theory.

It is interesting to note that Carl Friedrich Gauss may have anticipated such developments in mathematics already in 1816. The Paris Academy had proposed the proof or disproof of Fermat’s Last Theorem as its prize problem for the period. When attempts were made to persuade Gauss to compete, the Prince of Mathematicians replied: “Fermat’s Last Theorem is an isolated proposition which has little interest for me, because I can easily lay down a multitude of such propositions, which one can neither prove nor disprove.”

2 Recursive unsolvability

Dramatic as they may seem, the classical unsolvability proofs have nevertheless a distinctly relative [725] character. The problem of doubling the cube can be solved if the use of a ruler in the form of a right angle is permitted.

Similarly, the proofs of unprovability have their relative nature. There is no concept of a proposition undecidable per se, only that of a proposition undecidable on the basis of given axioms. The continuum hypothesis becomes provable if we supplement the axioms of set theory with Gödel’s axiom of constructibility [12]. Moreover, the presence of undecidable sentences in a theory is not particularly surprising unless we feel that the axioms in question ought to be categorical. Before the discovery that the parallel postulate was independent, it had been believed for centuries that Euclid’s axioms were a categorical description of space.

It is natural to ask if there are problems in some sense absolutely unsolvable. In the 1930’s, proofs of a different sort of undecidability appeared. For the first time mathematical problems were proven unsolvable in the sense that there is no finite algorithm for dealing with them. The early explorers were the logicians A. Church, K. Gödel, S. C. Kleene, E. Post, J. B. Rosser and A. Turing. Problems of the type then considered have come to be known as decision problems.

Suppose we are given a countable set W , with a fixed effective representation of its elements by finite strings, and a particular subset A of W . By a decision procedure for A we mean an algorithm, in the form of a finite list of instructions, which permits us to decide effectively, for each element of W , whether or not it is an element of the set A . The decision problem for the set A is the problem of finding such a decision procedure, or proving that no such procedure exists. The set A is said to be decidable or undecidable according as a decision procedure exists or not. In the case that it does not exist, one also says that the decision problem is recursively unsolvable.

The original researchers, mentioned previously, defined the exact concept of an algorithm in the 1930’s. This important work made recursive unsolvability proofs possible. Of course these researchers were, at that time, motivated primarily by decision problems in logic; questions of existence of mechanical procedures, like truth tables, for determining the truth or falsity of propositions. Today however, spurred by the advent of the large digital computer, questions of decidability have acquired much more than their former philosophical interest. The past quarter century has seen the solution

of important decision problems in nearly every branch of mathematics. Let us consider some examples.

One of the most important decision problems is the very difficult word problem for groups. Finitely presented groups arise in a natural way in algebraic topology, for example in the problem of classification of knots. A group is presented as a finite system of generators and relations. The word problem is the problem of deciding, given a word on the generators and their inverses, whether or not this word is equal to the identity, in consequence of the relations. (Equivalently, whether two words are equal.)

The word problem was one of the first decision problems to be formulated. It was first considered by M. Dehn and A. Thue in 1912–14. An important first step toward its eventual solution (in the [726] negative) was taken by E. Post and A. Markov in 1947. They (independently) proved the analogous word problem for semi-groups unsolvable (cf. [8, Chap. 6]). As seems to be the rule rather than the exception in this subject, the (negative) solution of the word problem for groups was also obtained nearly simultaneously in the East and West. During the period 1954–56, P. S. Novikov and W. W. Boone independently proved the word problem for groups undecidable. For groups the problem was much more difficult than for semi-groups. Novikov’s original paper [27] is 143 pages long. Boone’s proof, contained in a long series of six papers extending over four years, was later shortened by J. L. Britton. Britton’s proof is now available in a textbook [39].

Not only is the word problem for groups unsolvable in the general sense; Novikov and Boone produced particular groups with unsolvable word problem. Concerning this it is interesting to note that in consequence of the Higman, Neumann, Neumann embedding theorem [15], such groups must exist with only 2 generators.

Not all results on the word problem have been negative. For many groups, for example finite groups and free groups, the word problem is solvable. K. Reidemeister proved that the word problem for an Abelian group is solvable [33]. Other positive results may be found in [20].

The word problem concerns the elements of a group. More important to algebraists are decision problems concerning algebraic properties of groups as a whole. M. Rabin [31] and S. I. Adjan [1] (independently) obtained important results on these decision problems. They showed that for a wide class of algebraic properties, including finiteness, commutativity, cyclicity, simplicity, solvability and many others, there does not exist an effective method of deciding from presentations, whether the defined group has the property in question. This theorem led Rabin and Adjan to the negative solution of the isomorphism problem: There is no algorithm to decide, given two presentations, whether or not the groups defined by them are isomorphic.

The Rabin-Adjan theorem also applies to the property of having a solvable word problem. Thus in general we cannot tell from a presentation, whether the word problem is solvable or not. And oddly enough, it turns out that there is no single uniform partial algorithm which, from an arbitrary finite presentation and a word, solves the word problem for every presentation of a group whose word problem is solvable. This last result is proved in [5]. More information on decision problems in group theory may be found in [25].

Since the word problem was not on its face a problem in logic, its solution did much to convince mathematicians that the work of logicians could have significant consequences outside logic. This opinion was further reinforced in 1958 when an undecidability result appeared in topology. A. Markov proved [21] that the homeomorphism problem for 4-manifolds was undecidable. Roughly, this is the problem of deciding, given (suitable descriptions of) two 4-dimensional manifolds, whether or not they are homeomorphic (cf. [4]). The homeomorphism problem for 2-manifolds is solvable by well-known methods. At the time of writing (1974), the problem for 3-manifolds was still open.

Also in combinatorial geometry one finds decision problems: the so-called domino problem [727] (proved undecidable by Berger [2]) and various other problems of tiling the plane (cf. [37]). John Conway has announced undecidability results for prediction problems in his “Game of Life”.

Decision problems arise naturally even in analysis. In calculus one is confronted with the elementary integrability problem. Recall that a function of a real variable is elementary if it can be built up from integers, the rational functions, n th roots, exponentials, logarithms, trigonometric functions and their inverses, by addition, multiplication and composition. On intervals where the expression has regular, consistently chosen branches, its derivative is again elementary, but some elementary functions do not have elementary antiderivatives. For example, it is well known [35] that the following integrals are not elementary:

$$\int e^{-x^2} dx, \quad \int \frac{\sin(x)}{x} dx, \quad \int \sin(x^2) dx.$$

Thus it is natural to ask if there is an algorithm by means of which we may recognize whether a given elementary function is elementarily integrable or not. The answer depends on the precise expression class and its domain conventions. D. Richardson [34] proved undecidability results, including an integration problem, for suitably enlarged classes of real expressions allowing absolute value and a global real-domain interpretation. These qualifications are essential: this is not an unconditional impossibility theorem for the usual differential-algebraic formulation of elementary integration, for which decision procedures are available under suitable effective constant-field hypotheses.¹

Bound up with many results in logic is the important Entscheidungsproblem, the decision problem for provability of sentences of an axiomatic theory. Numerous important cases of this problem have now been solved, notably by A. Tarski [40], and not all of them in the negative. A celebrated theorem of Tarski [41] asserts that there is a decision procedure for deciding the truth or falsity of all first-order sentences of elementary algebra and geometry. Here the algebraic language has integer constants, equality, inequalities, addition and multiplication, with all quantified variables ranging over the real numbers.

The tenth problem on David Hilbert’s famous list [16] was a decision problem. Hilbert asked for an algorithm to decide of a polynomial equation, in several variables, with integer coefficients, whether or not the equation had solutions in integers. Ju. V. Matijasevič recently showed that this problem is unsolvable. Matijasevič proved that there is no such algorithm [22], [23].

Hilbert’s tenth problem has had a long and interesting history. Hilbert first stated the problem at the International Congress of Mathematicians in Paris in 1900. At that time, one could imagine only a positive solution to the problem. The concept of algorithm had not yet been worked out. Indeed, largely for this reason, significant work was not done on the problem until the 1950’s. At about this time, the early fifties, Julia Robinson, Martin Davis and Hilary Putnam began studying Diophantine sets. [A relation $R(x_1, x_2, \dots, x_n)$ is said to be Diophantine if there exists a polynomial $P(x_1, x_2, \dots, x_n, y_1, y_2, \dots, y_m)$, with integer coefficients, such that $R(x_1, x_2, \dots, x_n)$ holds if and only if integers $y_1, y_2, \dots, y_m$ exist for which $P(x_1, x_2, \dots, x_n, y_1, y_2, \dots, y_m) = 0$.] In 1961 Robinson, Davis and Putnam came very close to the negative solution of Hilbert’s tenth problem. They succeeded in showing that its unsolvability would follow from the existence of a single Diophantine predicate with exponential growth [10]. In 1970 Matijasevič was able to use this result to deliver the coup de grâce to Hilbert’s tenth problem. By means of clever number theoretic [728]

¹Editorial qualification: compare R. H. Risch, “The solution of the problem of integration in finite terms,” *Bull. Amer. Math. Soc.* 76 (1970), 605–608. This note is additional to the original numbered bibliography.

arguments, termed “elementary but ingenious” by J. W. S. Cassels, Matijasevič proved that the relation “ v is the 2 u th Fibonacci number” was Diophantine.

The solution of this long outstanding problem is one of the most significant events to occur in pure mathematics in the last decade. Only one byproduct of the ingenious solution of Hilbert’s tenth problem is the long sought polynomial formula for the prime numbers. From results of Putnam [29] and Matijasevič [22] [23] it follows that there exists a polynomial with integer coefficients whose positive values, on the prescribed domain of non-negative integer arguments, form exactly the set of prime numbers. (Negative integers also appear in the range.) In reference [24] Matijasevič has shown that 21 variables are sufficient in this polynomial.

It is interesting to consider variants of Hilbert’s tenth problem. We may enquire about existence of solutions in sets of numbers other than the integers. Of course for complex numbers the problem is decidable. An equation $P = 0$ over the complex numbers has a solution if and only if P is the zero polynomial or is nonconstant; the only insoluble case is a nonzero constant polynomial. Also for the set of real numbers, the problem is decidable. This fact is less obvious. The algorithm of Tarski for elementary algebra [41] provides a decision method in this case. For the integers, non-negative integers and positive integers, the problem is unsolvable [23], [9]. For rational numbers, however, the problem was still open at the time of writing (1974).

Unsolvability results have also appeared in game theory. M. Rabin proved in 1957 [30] that there exists a two-person win–lose game, with perfect information and decidable rules, in which neither player has a computable winning strategy. The author has also discovered an example of such a game (somewhat simpler than Rabin’s) [17]. We describe our Turing machine game in Section 7.

In considering recursive unsolvability results, it is important to keep in mind the very wide scope of the concept of algorithm. An algorithm, in the form of a finite list of instructions, is potentially a quite powerful problem-solving procedure. Today, computing machines, using algorithms, not only play games, but learn to play games. A computer can be programmed to play a better game of checkers than can be played by the person who wrote the program. Furthermore, the digital computer is the manmade analogue most closely resembling the brain. Thus an algorithmic unsolvability proof establishes unsolvability in a quite strong sense. Post has written [28]: Like the classic unsolvability proofs, these proofs are of unsolvability by means of given instruments. What is new is that in the present case these instruments, in effect, seem to be the only instruments at man’s disposal.

3 Turing machines

An algorithmic unsolvability proof is supposed to establish the impossibility of an algorithm. But the concept of algorithm is vague. Before it can be proved that something does not exist it must be defined precisely. What is an algorithm?

If we analyze the notion we see that it is closely bound up with the concept of computability. Consider a total function from the natural numbers $\mathbb{N} = \{0, 1, 2, \dots\}$ to the natural numbers. Intuitively, the function should be computable if there exists an algorithm permitting us to obtain [729] its values for each argument. Conversely, the decision problem for a set reduces to the computation problem for a certain function; the characteristic function of the set.

Thus we will have made our ideas precise once we succeed in defining the class of computable functions (of a natural number variable). What is meant exactly when we say that a function is computable?

If an algorithm exists for computing a function then it should be possible to program a computer to carry out the algorithm and calculate the value of the function. At least this should be possible in principle, allowing the computer sufficient time (an arbitrary finite amount) and sufficient external storage for intermediate calculations (again an arbitrary finite amount).

Today, one might attempt to base a definition of computability upon physically existing computers. However, this would be unsatisfactory. It would leave unanswered two very important questions. What is to be the exact relationship between the computer and its external memory (the machine-tape coupling)? And, more importantly, what operations need a computer be capable of in order to perform “all possible computations”?

For the purpose of defining computability one is forced to abandon physically existing machines. One must instead describe an idealized mathematical model of a computer.

This is what was done by A. Turing in a classic paper published in 1936 [42]. In this paper, Turing defined the class of theoretical computing machines that now bear his name. Here is specified in exact detail a simple mechanical basis sufficient to carry out any computation whatsoever.

We shall not reproduce here Turing’s original formulation in detail. Rather we present a simplification (due to Kleene [18] and Radó [32]). The essence of the conception is nevertheless Turing’s.

One visualizes a Turing machine in the form of a finite reading and writing device operating upon a two-way infinite linear tape. The tape is ruled off into squares. Upon each square of the tape may be printed a symbol from a finite alphabet, including “0” (blank) and “1”. At each moment of time, the machine is assumed to be in only one of a finite number of internal states, which we number $0, 1, 2, \dots, n$. Here $1, \dots, n$ are the active states; the halting state 0 is not counted in the phrase “ n -state machine”. The machine can scan only a single square of the tape at a time. Its behavior is completely determined by its current internal state and the contents of this scanned square. It erases the symbol on this square, prints a new one (possibly the same), shifts itself left or right on the tape (so as to scan an adjacent square), and changes itself into a new internal state (possibly the same state).

Turing believed that any computation could be reduced to a succession of such simple acts.

Turing machines operate sequentially, performing one act per unit of time. They are assumed to begin all computations initially in state 1 (the starting state). State 0 is the stopping state. The [730] machine is said to halt (cease operations) upon entering this inactive state.

It is more accurate to think of each individual Turing machine as a program rather than a machine. We represent this program with a finite set of “computer cards”. Each instruction card corresponds to an active state of the machine. When the alphabet is restricted to the two symbols “0” and “1”, this representation is particularly simple.

Example 1. A Turing machine with two active states.

State 1				State 2			
read	write	move	next	read	write	move	next
0	1	R	2	0	1	L	1
1	1	L	2	1	1	L	0

The i th card represents the i th state of the Turing machine. On each card, the first column gives the scanned symbol, the second column is the overprint by column, the third column is the shift right or left column and the fourth column gives the next state to enter.

Suppose the machine of example 1 is started in state 1 scanning the symbol 1, the remainder of the tape assumed blank. Then card 1 instructs it to leave this symbol unchanged, shift left on the tape and enter state 2. By assumption the square now scanned contains 0. Card 2 instructs the machine to overprint this 0 by 1, move left and (re)enter state 1. The scanned square is again 0 so card 1 instructs the machine to overprint this 0 by 1, move right and enter state 2. Now the machine sees 1, so card 2 instructs it to leave this 1 unchanged, shift left and stop. The result is three ones printed on the tape.

One could represent the machine's behavior with the sequence of successive tape conditions:

$$\begin{array}{c} 001 \\ 1 \end{array} \longrightarrow \begin{array}{c} 001 \\ 2 \end{array} \longrightarrow \begin{array}{c} 011 \\ 1 \end{array} \longrightarrow \begin{array}{c} 111 \\ 2 \end{array} \longrightarrow \begin{array}{c} 111 \\ 0 \end{array}.$$

In this display, the subscript beneath a tape symbol gives the current state and identifies the scanned square; all unshown squares are blank. Apparently, the effect of this machine is to "add 2". If this machine is started in state 1 scanning the leftmost of $n \geq 1$ consecutive ones, with the rest of the tape blank, then it will be seen to halt after 4 shifts. And when it does, it will be scanning the leftmost of $n + 2$ consecutive ones.

We represent the natural number x on the tape with a string of $x + 1$ consecutive ones (tallies). The number 0 is represented on the tape as a single 1 to distinguish it from a blank square. An ordered pair could be represented with two groups of tallies separated by a blank (0).

With these conventions we may now define computability for functions of a natural number.

Definition. A total function $f : \mathbb{N} \rightarrow \mathbb{N}$ is Turing computable if there exists a Turing machine [731] M such that, for every $x \in \mathbb{N}$, when M is started in state 1 scanning the leftmost of exactly $x + 1$ consecutive ones on an otherwise blank tape, M eventually halts scanning the leftmost of exactly $f(x) + 1$ consecutive ones, with every other tape square blank.

In this sense the machine of example 1 computes the function $f(x) = x + 2$. The following Turing machine computes the function $f(x) = 2x$.

Example 2. A Turing machine computing $f(x) = 2x$.

State 1				State 2				State 3				State 4			
read	write	move	next	read	write	move	next	read	write	move	next	read	write	move	next
0	0	R	4	0	1	R	3	0	1	R	1	0	0	L	4
1	0	L	2	1	1	L	2	1	1	R	3	1	0	L	5

State 5			
read	write	move	next
0	0	R	0
1	1	L	5

It is important to note that although the amount of information stored at any one time by a Turing machine (on its tape) is always finite, we do not place any preassigned upper bound on this amount. To do so would be tantamount to making our definition dependent upon computer technology and therefore mathematically uninteresting.

4 Church's thesis

Is every intuitively effectively computable function computable by a Turing machine? The answer to this question appears to be “yes”. This may be difficult for the reader to believe. Perhaps, since computation is normally done on sheets of paper, it might be thought that the two-dimensional nature of paper is essential. However, finite portions of a two-dimensional array, together with their coordinates and symbols, can be encoded effectively by finite strings on a one-dimensional tape. A fixed finite alphabet does not require each entire sheet to fit into one tape square. Also, our insistence that there be only a finite number of symbols in the alphabet is not a severe restriction. If more symbols are needed it is always possible to use sequences of symbols in place of individual symbols. (Indeed, for this reason, we may restrict our machines to the use of only two symbols “0” and “1”. It is well known that the class of functions computed is not diminished by so doing. For a proof of this see [26, p. 129].)

The assertion that every intuitively effectively computable function is Turing computable is known as the Church-Turing thesis. It is called a “thesis” rather than a “theorem” because it is not susceptible to proof. The thesis is rather in the nature of a definition. It is a proposal that we identify our intuitive concept of computable function with the exact mathematical substitute (Turing computability).

Turing believed that any process which could effectively be carried out, could actually be realized on one of his machines. And the years have tended to bear out Turing's view. Today, the Church-Turing thesis is generally accepted by virtually all workers in logic.

Convincing evidence for the Church-Turing thesis is provided by certain historical events. Recall [732] that Turing put forward his proposed characterization of computability in 1936 [42]. At about this same time, two other characterizations were also proposed: The λ -definable functions (Church, Kleene), and the general recursive functions (Herbrand, Gödel). Superficially, the three formulations appeared to be different. Between them they encompassed all methods of calculation known at the time, including the various definitions by recursion (induction). Yet the three different characterizations were proved equivalent. The same (evidently fundamental) class of functions is obtained in each case.

An additional argument in favor of the Church-Turing thesis is given by Kleene [18]: A great stock of intuitively computable functions (all that have been investigated in this connection) are known to be Turing computable. Likewise we have a great stock of methods or operations (for obtaining new intuitively computable functions from others) which are paralleled by operations for building new Turing machines from given Turing machines (or the analog in terms of recursiveness). If there were a function which is intuitively computable but not Turing computable, it would have to be “inaccessible” by any process of building up toward it from this stock of functions and operations already mastered.

So it was that the recursive functions were discovered. Concerning this class of functions Gödel remarks [11]: “... with this concept of recursiveness one has for the first time succeeded in giving an absolute definition of an interesting epistemological notion, i.e., one not depending upon the formalism chosen.” Post has remarked: “Indeed, if general recursive function is the formal equivalent of effective calculability, its formulation may play a role in the history of combinatory mathematics second only to that of the formulation of the concept of natural number.” [28].

5 A non-computable function

The machine presented in Example 1 is one of the highest scorers in a game invented by T. Radó [32]. Radó’s “Busy Beaver n game” is the problem of programming an n -state (2-symbol) Turing machine to print the largest possible number of ones onto a (blank) tape and halt. If the machine of example 1 is started in state 1 on a blank tape it will be seen to halt after 6 shifts and when it does there will be 4 ones printed on its tape. 4 is the highest score obtainable in the Busy Beaver 2 game.

What is the highest score possible in the Busy Beaver n game? For $n \geq 1$, a 2-symbol Turing machine of n active states is formally a mapping

$$\delta : \{0, 1\} \times \{1, 2, 3, \dots, n\} \rightarrow \{0, 1\} \times \{R, L\} \times \{0, 1, 2, 3, \dots, n\}.$$

Thus there are exactly $(4n + 4)^{2n}$ fully specified, labelled transition tables with n active states, a finite number. Unreachable states and transitions are permitted, and are counted. Some of these will fail to halt if started on a blank tape; the machine of example 2 loops forever. However, among the machines which halt, there must be one or more which print the maximum possible number of ones. Let $\Sigma(n)$ denote this maximum score.

Evidently, Σ is a well defined function of n , because the finite set of scores of halting machines is nonempty and hence has a greatest element. An immediate-halting transition supplies at least one [733] such machine. The function Σ has been calculated for a few small values of n . It is known that $\Sigma(1) = 1$ and $\Sigma(2) = 4$. S. Lin proved in [19] that $\Sigma(3) = 6$. Also, one sees immediately that for each n , $\Sigma(n) < \Sigma(n + 1)$. For, with an additional card, a machine may be instructed to move right (or left) across ones to overprint with 1 the first 0 encountered, and halt.

State $n + 1$			
read	write	move	next
0	1	R	0
1	1	R	$n + 1$

Redirect each old transition to the halting state into state $n + 1$ instead. The added state scans right across any ones and changes the first zero to a one before halting.

Can a formula for $\Sigma(n)$ be found? There are difficulties in determining $\Sigma(n)$ for large values of n . First of all, there is an enormous number of machines to consider. There are 25,600,000,000 different Turing machines with 4 states. Of course, many of these fail to halt, and may therefore be eliminated from consideration. However, it is very difficult to decide, given a Turing machine, whether or not it will eventually halt. This is the so-called halting problem. We shall see that the function Σ is not computable (non-recursive).

Now, whether Σ were computable or not, one would nevertheless expect that it would be possible to give an upper bound for Σ . One would expect to be able to prove that $\Sigma(n) \leq 2^n$ or perhaps $\Sigma(n) \leq n^{(n^n)}$ or at least some such inequality. Surprisingly enough, no such inequalities hold! These two proposed bounds are computable functions. Not only is Σ non-computable, Radó discovered [32] that Σ has no computable upper bound.

Theorem 1. Let f be any increasing computable function. Then for n sufficiently large, $f(n) < \Sigma(n)$. In particular Σ is non-computable.

The proof is almost trivial. Let M be a Turing machine with c states which computes f . Denote by T the Turing machine of 5 states which computes the function $2x$ (example 2).

For each integer $x \geq 1$, let $M^{(x)}$ be a Turing machine with $x + 1$ states which will print $x + 1$ consecutive ones onto a blank tape and halt scanning the leftmost of these. For example, $M^{(2)}$ could be taken to be the following machine:

State 1				State 2				State 3			
read	write	move	next	read	write	move	next	read	write	move	next
0	1	L	2	0	1	L	3	0	1	R	3
1	1	L	0	1	1	L	0	1	1	L	0

The read-1 rows in states 1 and 2 are unreachable on blank input; their entries above are arbitrary completions of rows left blank in the original. For general $x \geq 1$, states $1, \dots, x$ write a one on zero, move left, and enter the next state. State $x + 1$ writes a one on zero and moves right in the same state; on one it leaves the one unchanged, moves left, and halts. All otherwise unused entries can be filled arbitrarily.

Now consider the composition of machines, which we denote by

[734]

$$M \left[T \left[M^{(x)} \right] \right].$$

It is obtained by relabeling the active states into disjoint blocks and redirecting each intermediate machine's halt transitions to the starting state of the next machine. By the clean unary input/output convention, it is constructed so as to print $f(2x) + 1$ ones onto a blank tape and halt. Since this composite machine has $x + 6 + c$ states it follows that

$$f(2x) < \Sigma(x + 6 + c). \quad (i)$$

Now for $x \geq 6 + c$ we have $x + 6 + c \leq 2x$. Therefore, since f is assumed to be increasing,

$$f(x + 6 + c) \leq f(2x). \quad (ii)$$

Let $n = x + 6 + c$. Then for $n \geq 12 + 2c$ we have $f(n) < \Sigma(n)$ by (i) and (ii). Since Σ itself is increasing we have proved that Σ is non-computable.

Corollary 1. Let f be any computable function (increasing or not). Then for n sufficiently large, $f(n) < \Sigma(n)$.

Proof. For each computable function f we can find an increasing computable function $\hat{f}$ such that $f(n) \leq \hat{f}(n)$. For example we may take

$$\hat{f}(n) = n + \sum_{i=0}^n f(i).$$

Intuitively, $\hat{f}$ is computable. (For a rigorous proof of this see [18, p. 224].)

6 The halting problem

Evidently, the only obstacle to the computation of Σ is the halting problem: to decide, given a Turing machine M , whether or not M will eventually halt when started in state 1 on a blank tape. The foundational undecidability results were obtained by Church [6] and Turing [42] in 1936–37; their original formulations should be distinguished from this particular blank-tape formulation.

An informal proof of the undecidability of the halting problem may be given, based on the non-computability of Σ . Assume for contradiction that an algorithm A existed for deciding the halting problem. Then Σ could be computed algorithmically as follows: Given n , list the $(4n + 4)^{2n}$ n -state Turing machines. Using A , strike out from the list those which will not halt. Operate the others until they halt. Examine the number of ones on their tapes for the largest. Since Σ is not computable this is a contradiction. (For a rigorous proof see [18].)

For programming, the fact that the halting problem is recursively unsolvable means that there does not exist one program capable of determining of an arbitrary program whether or not that arbitrary program will eventually halt.

7 An unsolvable game

Consider the following two-person win–lose game with perfect information. We shall refer to it as the Turing machine game.

The Turing machine game. There are two players, I and II. They take turns choosing positive [735] integers:

first	player I picks n ;
then	player II picks m , knowing n ;
finally	player I picks k , knowing n and m .

Player I wins if some n -state Turing machine halts in exactly $m + k$ shifts when started on a blank tape. Otherwise, player II wins.

In contrast to many games considered in the theory of games, the Turing machine game may conceivably be played. It is possible to decide, after the game is over, who has won. After I and II have selected their integers, a referee need only list the $(4n + 4)^{2n}$ n -state Turing machines and then simulate each of them for at most $m + k$ shifts, stopping a simulation upon halting, and check whether any first halt occurs on shift $m + k$. It is not necessary for the referee to be able to solve the halting problem. A machine can be programmed to decide the winner.

Now it is well known that a finite-length, perfect-information win–lose game with no draws is determined. There exists a winning strategy for one of the players. In this case it is player II. However, we shall show that the Turing machine game is nevertheless not trivial to play. In this game neither player has an algorithm for winning.

Theorem 2. In the Turing machine game neither player has a computable winning strategy.

Proof. Let $\text{SH}(n)$ be the maximum possible number of shifts which an n -state Turing machine can perform before halting (after being started in state 1 on a blank tape). Obviously $\Sigma(n) \leq \text{SH}(n)$. A strategy for player II is a function $m = f(n)$. Plainly, f is a winning strategy for player II if and

only if $\text{SH}(n) \leq f(n)$. Thus player II has a winning strategy $\text{SH}(n)$. However, by Corollary 1, for each computable function f , eventually $f(n) < \Sigma(n) \leq \text{SH}(n)$. Hence player II has no computable winning strategy. Since player II has a winning strategy, player I has no winning strategy at all, and in particular no computable one.

8 Generalizations of the Busy Beaver problem

We have defined $\text{SH}(n)$ to be the maximum possible number of shifts which an n -state (2-symbol) Turing machine can perform before halting, when started in state 1 on a blank tape. Only machines that halt enter this maximum. Analogously define $\text{SC}(n)$ to be the maximum possible number of tape squares which an n -state Turing machine can scan in an active state before halting (when started in state 1 on a blank tape). The square entered by the final halting shift is not counted unless it was scanned earlier in an active state. Define $H(n)$ to be the number of different labelled, fully specified n -state Turing machines which halt when started in state 1 on a blank tape. Plainly,

$$\Sigma(n) \leq \text{SC}(n) \leq \text{SH}(n), \quad (1)$$

$$H(n) < (4n + 4)^{2^n}. \quad (2)$$

From these inequalities it follows that the functions SC and SH are non-computable. The function H is also non-computable. Indeed, given $H(n)$, one can dovetail all n -state machines until exactly $H(n)$ of them have halted. These are all the halting machines; the rest never halt. Thus an algorithm for H would solve the halting problem. The values and lower bounds reported in 1974 [736] appear in Table 1.

Table 1: Values and lower bounds reported in 1974 (not a current record table).

n	$\Sigma(n)$	$\text{SC}(n)$	$\text{SH}(n)$	$H(n)$
1	1	1	1	32
2	4	4	6	9,784
3	6	7	21	7,571,840
4	≥ 13	≥ 14	≥ 107	—
5	≥ 16	—	—	—
6	≥ 35	—	≥ 436	—
7	$\geq 22,961$	—	$\geq 10^{693}$	—
8	$\geq 9 \times 10^{41}$	—	—	—
10	$\geq 10^{2 \times 10^{44}}$	—	$\geq 10^{4 \times 10^{44}}$	—

An em dash means that no entry was supplied in the original table. It does not mean zero, nor does it assert that a value is currently unknown. The original table has no column for $n = 9$.

Results in Table 1 are due to T. Radó, S. Lin, C. Y. Lee, P. Fischer, M. Green, D. Jefferson, J. Slater and the author.

As was already evident from Theorem 1, the rate of growth of the function Σ is astronomical. This is illustrated in the $\Sigma(n)$ column by the very large scores in the Busy Beaver 8 and 10 games (achieved by Green [14]). Not only is it true that $\Sigma(n) < \Sigma(n + 1)$ for each n . Indeed it is easy to see that for any computable function f

$$f(\Sigma(n)) < \Sigma(n+1)$$

for infinitely many values of n . Thus, for example, infinitely often we have

$$\Sigma(n)! < \Sigma(n+1).$$

The functions SC and SH are related to Σ . We can prove

$$\text{SC}(n) < \Sigma(3n), \tag{3}$$

$$\text{SH}(n) \leq n \text{SC}(n) 2^{\text{SC}(n)}. \tag{4}$$

Inequality (3) follows by simulating a halting n -state machine on every other tape square and marking the intervening squares with ones. The construction can be arranged with at most $3n$ active states to leave strictly more ones than the number of actively scanned squares of the original machine. Unused states may be added to obtain exactly $3n$ states. A detailed construction justifying both the strict inequality and the state bound is given in the editorial notes, `Papers/1974/jones1974_editorial_notes.md` (entry 1974-59), and is formalized in `Lean/Diophantine/Paper1974/Doubling.lean`. The inequality (4) is derived by counting tape-state configurations. These inequalities are due to T. Radó, S. Lin and the author. It is not difficult to show, using (3) and (4), that all four non-recursive functions have the same degree of recursive unsolvability ($0'$). The range of the function Σ is an example of a hyperimmune set. The range of H is retraceable and nonrecursive, hence immune. It is not hyperimmune, because its increasing enumeration $H(n)$ has the computable upper bound $(4n+4)^{2n}$. The set $\{(m, n) : m \leq H(n)\}$ is r.e. nonrecursive.

Editorial proof detail. Write $U_n = (4n+4)^{2n}$ and $L_n = 4(4n+4)^{2n-1}$. The machines which halt on their first blank-tape transition already give $L_n \leq H(n)$, and (2) gives $H(n) < U_n$. For $n \geq 2$ one has $L_n > U_{n-1}$, so these computable, disjoint intervals determine n from a value $H(n)$. Dovetail the n -state machines until that many have halted; their complete halting table then determines which canonically padded $(n-1)$ -state machines halt, and therefore determines $H(n-1)$. This is a partial computable retracing procedure (with $H(1)$ mapped to itself). If the range had an infinite r.e. subset, for any n one could wait for a member above U_n and retrace to $H(n)$, making H computable. This proves immunity; the same interval bounds also prove that H is strictly increasing.

Consider a Turing machine programmed by a monkey. For each fixed n , choose uniformly among the $(4n+4)^{2n}$ labelled transition tables. The probability of halting on a blank tape is then $H(n)/(4n+4)^{2n}$. One may ask whether these probabilities have a limit as n tends to infinity:

$$\lim_{n \rightarrow \infty} \frac{H(n)}{(4n+4)^{2n}}.$$

From Table 1 we find that the first three values of this quotient are (approximately) 0.500, 0.472 [737] and 0.451, rounded to three decimal places. The existence and value of the displayed limit are not established here.

Acknowledgment. The author wishes to thank Professors Verena H. Dyson and Roger C. Lyndon for many helpful suggestions regarding the preparation of this paper.

References

- [1] S. I. Adjan, The algorithmic unsolvability of problems concerning certain properties of groups (Russian), *Dokl. Akad. Nauk SSSR*, 103 (1955) 533–535.
- [2] R. Berger, The undecidability of the domino problem, *Mem. Amer. Math. Soc.*, no. 66, (1966).
- [3] W. W. Boone, Certain simple unsolvable problems of group theory, *Indag. Math.*, 16 (1954) 231–237, 492–497; 17 (1955) 252–256, 571–577; 19 (1957) 22–27, 227–232.
- [4] W. W. Boone, W. Haken and V. Poénaru, On recursively unsolvable problems in topology and their classification, *Contributions to Mathematical Logic. Proceedings of the logic colloquium, Hannover, Germany 1966*. Edited by H. A. Schmidt, K. Schütte and H. J. Thiele. North-Holland, Amsterdam, 1968.
- [5] W. W. Boone and H. Rogers Jr., On a problem of J. H. C. Whitehead and a problem of Alonzo Church, *Math. Scand.*, 19 (1966) 185–192.
- [6] A. Church, An unsolvable problem of elementary number theory, *Amer. Jour. Math.*, 58 (1936) 345–363.
- [7] P. Cohen, The independence of the continuum hypothesis, *Proc. National Acad. Sciences, U. S. A.*, 50 (1963) 1143–1148 and 51 (1964) 105–110.
- [8] M. Davis, *Computability and Unsolvability*, McGraw–Hill, New York, 1958, xxv + 210 pp.
- [9] M. Davis, Hilbert’s tenth problem is unsolvable, *American Mathematical Monthly*, 80 (1973) 233–269.
- [10] M. Davis, H. Putnam and J. Robinson, The decision problem for exponential Diophantine equations, *Ann. Math.*, 74 (1961) 425–436.
- [11] K. Gödel, Remarks before the Princeton bicentennial conference on problems in mathematics 1946. Reprinted in Davis, *The Undecidable*, Raven Press, Hewlett, N. Y., 1965, pp. 84–88.
- [12] K. Gödel, The Consistency of the Axiom of Choice and of the Generalized Continuum-Hypothesis with the Axioms of Set Theory, *Annals of Math. Studies*, no. 3 (1940), Princeton University Press, Princeton, N. J., 66 pp. Second printing 1951, v + 69 pp.
- [13] K. Gödel, Über formal unentscheidbare Sätze der Principia Mathematica und verwandter Systeme I, *Monatsh. Math. und Physik*, 38 (1931) 173–198. English translations: (1) On formally undecidable propositions of Principia Mathematica and related systems, Martin Davis (editor) *The Undecidable*, Raven Press, 1965, pp. 5–38. (2) Jean van Heijenoort (editor), *From Frege to Gödel*, Harvard University Press, 1967, pp. 596–616.
- [14] M. Green, *Proceedings of the Fifth Annual Symposium on Switching Circuit Theory and Logical Design*, Princeton University Conference, Oct. 1964, pp. 91–94.
- [15] G. Higman, B. H. Neumann and H. Neumann, Embedding theorems for groups, *J. London Math. Soc.*, 24 (1949) 247–254.
- [16] D. Hilbert, *Mathematische Probleme*, Vortrag, gehalten auf dem internationalen Mathematiker-Kongress zu Paris 1900. *Nachrichten Akad. Wiss. Göttingen, Math.-Phys. Kl.* (1900) 253–297. English translation: *Bull. Amer. Math. Soc.*, 8 (1901–1902) 437–479.

- [17] J. P. Jones, A simple unsolvable game, Abstracts of the IV-th International Congress for Logic, Methodology and Philosophy of Science, Bucharest, Rumania, 1971.
- [18] S. C. Kleene, Introduction to Metamathematics, Van Nostrand, Princeton, N. J., 1952, x + 550 pp.
- [19] S. Lin and T. Radó, Computer studies of Turing machine problems, J. Assoc. Comp. Mach., 12 [738] (1965) 196–212.
- [20] R. C. Lyndon, On Dehn's algorithm, Math. Ann., 166 (1966) 208–228.
- [21] A. Markov, Unsolvability of the problem of homeomorphy, Proc. Int. Cong. Math., Edinburgh 1958, pp. 300–306.
- [22] Ju. V. Matijasevič, Enumerable sets are Diophantine, Dokl. Acad. Nauk SSSR, 191 (1970) 279–282. English translation: Soviet Math. Doklady, 11 (1970) 354–357.
- [23] Ju. V. Matijasevič, Diophantine representation of enumerable predicates, Izvestija Akademii Nauk SSSR. Serija Matematičeskaja, 35 (1971) 3–30. English translation: Mathematics of the USSR–Izvestija, 5 (1971) 1–28.
- [24] Ju. V. Matijasevič, Diophantine representation of the set of prime numbers, Dokl. Akad. Nauk SSSR, 196 (1971) 770–773. English translation: Soviet Math. Doklady, 12 (1971) 249–254.
- [25] C. F. Miller III, On group-theoretic decision problems and their classification, Annals of Math. Studies, 68 (1971), Princeton, N. J., viii + 106 pp.
- [26] M. Minsky, Computation: Finite and Infinite Machines, Prentice-Hall, Englewood Cliffs, N. J. 1967, xvii + 317 pp.
- [27] P. S. Novikov, On the algorithmic unsolvability of the word problem in group theory, Akademiya Nauk SSSR, Matematicheskii Institut Trudy no. 44 (1955), Moscow. Amer. Math. Soc. Translations, ser. 2, 9 (1958) 1–122.
- [28] E. Post, Recursively enumerable sets of positive integers and their decision problems, Bull. Amer. Math. Soc., 50 (1944) 284–316.
- [29] H. Putnam, An unsolvable problem in number theory, J. Symbolic Logic, 25 (1960) 220–232.
- [30] M. Rabin, Effective computability of winning strategies, Annals of Math. Studies, Princeton, 39 (1957) 147–157.
- [31] M. Rabin, Recursive unsolvability of group theoretic problems, Ann. of Math., 67 (1958) 172–194.
- [32] T. Radó, On non-computable functions, Bell System Technical Jour., 41 (1962) 877–884.
- [33] K. Reidemeister, Einführung in die Kombinatorische Topologie, Braunschweig, 1932. (Also Chelsea, New York, 1950.)
- [34] D. Richardson, Some undecidable problems involving elementary functions of a real variable, J. Symbolic Logic, 33 (1968) 514–520.
- [35] J. F. Ritt, Integration in Finite Terms, Columbia University Press, New York, 1948.

- [36] J. Robinson, Hilbert's tenth problem, AMS Proceedings of Symposia in Pure Mathematics, 20 (1971) 191–194.
- [37] R. M. Robinson, Undecidability and nonperiodicity for tilings of the plane, Invent. Math., 12 (1971) 177–209.
- [38] H. Rogers Jr., Theory of Recursive Functions and Effective Computability, McGraw–Hill, New York, 1967, xix + 482 pp.
- [39] J. Rotman, The Theory of Groups: An Introduction, Allyn and Bacon, Boston, 2nd ed., 1973.
- [40] A. Tarski, A. Mostowski and R. M. Robinson, Undecidable Theories, North-Holland, Amsterdam, 1953, ix + 98 pp.
- [41] A. Tarski, A Decision Method for Elementary Algebra and Geometry, Univ. of California Press, Berkeley, California, 1951.
- [42] A. Turing, On computable numbers, with an application to the Entscheidungsproblem, Proc. London Math. Soc. ser. 2, 42 (1936–7) 230–265; corrections, *ibid.*, 43 (1937) 544–546.

Original affiliation: Department of Mathematics, Statistics and Computing Science, University of Calgary, Calgary, Alberta, Canada.